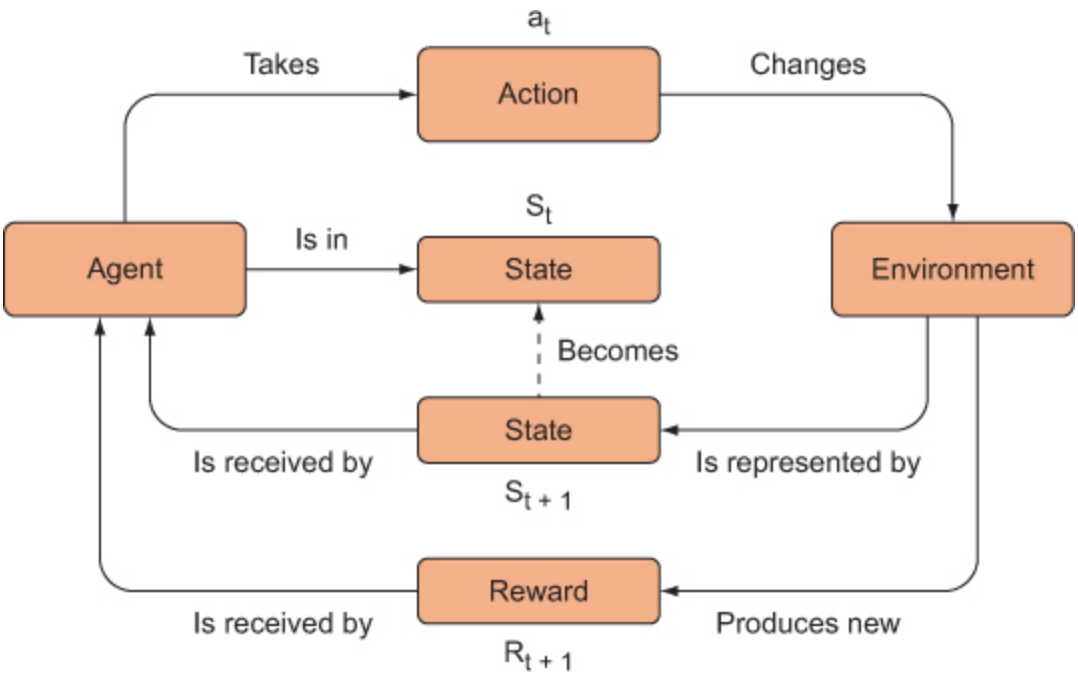
reward of +10 (an arbitrary value) whenever its action reduces the error value. Or more reasonably, we might grant a reward proportional to how much it decreases the error. If it increases the error, we would give it a negative reward.

Lastly, let's give our learning algorithm a fancier name, calling it the *agent*. The agent is the action-taking or decision-making learning algorithm in any RL problem. We can put this all together as shown in figure 1.8.

Figure 1.8. The standard framework for RL algorithms. The agent takes an action in the environment, such as moving a chess piece, which then updates the state of the environment. For every action it takes, it receives a reward (e.g., +1 for winning the game, –1 for losing the game, 0 otherwise). The RL algorithm repeats this process with the objective of maximizing rewards in the long term, and it eventually learns how the environment works.



In our data center example, we hope that our agent will learn how to decrease our cooling costs. Unless we're able to supply it with complete knowledge of the environment, it will have to employ some degree of trial and error. If we're lucky, the agent might learn so well that it can be used in different environments than the one it was originally trained in. Since the agent is the learner, it is implemented as some sort of learning algorithm. And since this is a book about *deep* reinforcement learning, our agents will be implemented using *deep learning* algorithms (also known as *deep neural*